AGENTIC AI WORKSHOP · DAY 6 OF 10

Model Context Protocol (MCP)

The knowledge base stops being code inside Meridian and becomes an MCP server any agent, in any language, can use.

You build: **MCP server** · about 3.4 h of material

Contents

1. One capability, many agents — Why this matters · 5 min
2. Host, client, server — Theory · 10 min
3. The 2026 spec, and why boards care — Theory · 8 min
4. Lab 1: Build the Northstar MCP server — Lab · beginner · 35 min
5. Lab 2: Inspect it like Postman — Lab · beginner · 20 min
6. Lab 3: Same agent, standard boundary — Lab · intermediate · 30 min
7. Typed results, annotations and errors — Theory · advanced · 8 min
8. Asking the person mid-tool: elicitation — Theory · advanced · 8 min
9. Lab 4: Typed tools, a person in the loop, and tests — Lab · advanced · 30 min
10. Securing MCP servers (and using other people's safely) — Theory · advanced · 10 min
11. Lab 5: Who's calling, and what may they do? — Lab · advanced · 25 min
12. MCP in production — Theory · advanced · 5 min
13. Check yourself — Quiz · 5 min
14. Day 6 recap — Recap · 3 min
15. Quiz answer key

WHY THIS MATTERS 5 MIN

One capability, many agents

Yesterday's search_policies is a Python function only Meridian can call. Now everyone wants it.

The Disputes team wants policy search in **their** agent (a different framework). Compliance wants it in **Claude Desktop**. The Java platform team wants it on **Day 10**. Without a standard, that's a custom integration for every pair: **N agents × M tools**. MCP turns it into **N + M**: build the server once, and every MCP-capable agent can use it.

By the end of today you can

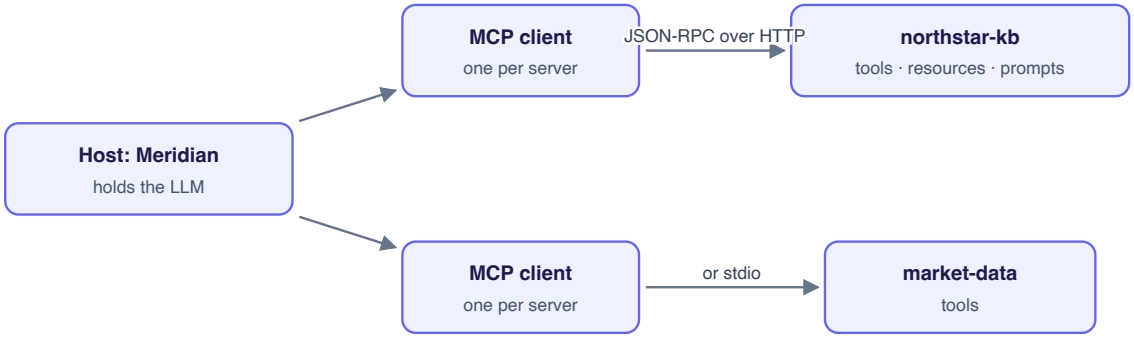
- Explain MCP's host / client / server model and its three primitives
- Know what changed in the 2026 spec, and why it matters to an architecture board
- Build an MCP server for the Northstar knowledge base
- Test it with MCP Inspector
- Swap Meridian's built-in tool for an MCP client: same agent, standard boundary

THEORY 10 MIN

Host, client, server

MCP is to AI tools what USB-C is to peripherals: one standard plug.

MCP architecture



1. The HOST is the AI app (Meridian, Claude Desktop, an IDE). It holds the model.
2. For each server it connects to, the host runs an MCP CLIENT.
3. Clients talk JSON-RPC to SERVERS: over Streamable HTTP for shared services, or stdio for local ones.
4. Tools are discovered at run time (tools/list), so new servers plug in without code changes.

Primitive	Controlled by	Example
Tools	the model (it decides to call)	`search_policies(question)`
Resources	the app (it decides what to attach)	`northstar://docs/merchant-risk-policy-2026`
Prompts	the user (picks a template)	`limit_report(merchant_id)`

MCP doesn't replace function calling. The host still turns MCP tools into the model's native tool calls. MCP standardises the boundary between the agent and the capability.

THEORY 8 MIN

The 2026 spec, and why boards care

MCP grew up: stateless, safer auth, and neutral governance.

Change (spec 2026-07-28)	Why it matters
Stateless core: no handshake, no sessions	MCP servers scale like any stateless HTTP service behind a load balancer
Issuer (`iss`) validation for OAuth	closes the "mix-up" attack, where a malicious server steals another server's code
Enterprise-managed authorization	the company identity provider decides which agents may reach which servers
Tasks extension	long-running jobs (a month-end review) don't have to finish in one HTTP request
12-month deprecation policy	upgrades you can plan, which regulated firms need

Governance: MCP (and, since Aug 2026, A2A) are run by the **Agentic AI Foundation** under the Linux Foundation. That's the answer when an architecture board asks "is this a vendor protocol?"

> Tutorials from 2025 use the v1 SDK. In SDK v2, **FastMCP is now MCPServer** (`from mcp.server.mcpserver import MCPServer`), and transport options moved to `run()`.

LAB BEGINNER 35 MIN

Lab 1: Build the Northstar MCP server

Goal. Expose the knowledge base as an MCP server: two tools, document resources and a prompt.

1. The server

Type hints become the schema; docstrings become the descriptions the model reads. `search_policies` reuses your Day 5 agentic RAG.

northstar_mcp/server.py

```
"""Day 6 Lab 1: Northstar knowledge base as an MCP server (SDK v2, Streamable HTTP).

Run (from the meridian/ root):  python -m northstar_mcp.server        -> http://127.0.0.1:8000/mcp
Inspect:                         npx @modelcontextprotocol/inspector   (Streamable HTTP, that URL)
"""
import os
import pathlib

from mcp.server.mcpserver import Context, MCPServer
from mcp.server.mcpserver.exceptions import ResourceError, ToolError

from data import MERCHANTS

DOCS = pathlib.Path(__file__).resolve().parent.parent / "docs"

mcp = MCPServer(
    "northstar-kb",
    title="Northstar Pay Knowledge Base",
    description="Merchant risk policies, SOPs, chargeback/reserve rules, API limits and pricing for
Northstar Pay (fictional).",
    version="1.0.0",
)


# ---------- resources (application-controlled context) ----------
@mcp.resource("northstar://docs")
def list_docs() -> list[str]:
    """List available Northstar documents."""
    return sorted(p.stem for p in DOCS.glob("*.md"))


@mcp.resource("northstar://docs/{name}")
def read_doc(name: str) -> str:
    """Read a Northstar policy / SOP / pricing document by name (e.g. merchant-risk-policy-2026)."""
    path = DOCS / f"{name}.md"
    if not path.exists():
        raise ResourceError(f"unknown document '{name}'. Available: {list_docs()}")
    return path.read_text()


# ---------- tools (model-controlled actions) ----------
@mcp.tool()
async def search_policies(question: str, ctx: Context) -> str:
    """Search Northstar Pay's internal merchant risk policies, SOPs, chargeback/reserve rules, API
limits and pricing.
    Returns a cited answer, or an explicit 'not found'. Superseded policies are excluded."""
    from agentic_rag import rag_graph  # Day 5 Lab 3 (imported lazily: needs the Chroma index +
embeddings)

    await ctx.report_progress(0.1, 1.0, "retrieving")
```

```
    out = await rag_graph.ainvoke({"question": question, "query": question, "docs": [], "retries": 0,
                                   "grade": None, "answer": ""})
    await ctx.report_progress(1.0, 1.0, f"done after {out['retries']} retries")
    return out["answer"]


@mcp.tool()
def get_merchant_risk_profile(merchant_id: str) -> dict:
    """Get a merchant's risk tier, MCC, chargeback ratio, rolling reserve % and watchlist/KYB
status."""
    m = MERCHANTS.get(merchant_id.strip().upper())
    if m is None:
        raise ToolError(f"unknown merchant; valid: {sorted(MERCHANTS)}")   # ToolError: the model sees
this message
    return {"merchant_id": merchant_id.upper(), "risk_tier": m["risk_tier"], "mcc": m["mcc"],
            "chargeback_ratio_pct": m["chargeback_ratio_pct"], "rolling_reserve_pct":
m["rolling_reserve_pct"],
            "kyb_notes": m["kyb_notes"]}


# ---------- prompts (user-controlled templates) ----------
@mcp.prompt()
def limit_report(merchant_id: str) -> str:
    """Draft a processing-limit notification for the risk committee."""
    return (f"Prepare a processing-limit notification for {merchant_id}: MTD volume vs limit (in limit
currency), "
            "utilisation %, required approver for any increase per Merchant Risk Policy §3, and next
steps. Cite policy sections.")


if __name__ == "__main__":
    mcp.run(transport="streamable-http", host=os.getenv("MCP_HOST", "127.0.0.1"),
port=int(os.getenv("MCP_PORT", "8000")))
```

2. Run it

terminal

```
pip install "mcp[cli]>=2.3,<3"
mkdir -p northstar_mcp && touch northstar_mcp/__init__.py   # then save server.py inside it
python -m northstar_mcp.server        # serves http://127.0.0.1:8000/mcp
```

3. Smoke-test with the SDK client

In a second terminal:

smoke.py

```
import asyncio
from mcp import Client

async def main():
    async with Client("http://127.0.0.1:8000/mcp") as c:        # URL → Streamable HTTP
        print([t.name for t in (await c.list_tools()).tools])
        r = await c.call_tool("get_merchant_risk_profile", {"merchant_id": "CRUX"})
        print(r.structured_content)
        r = await c.call_tool("search_policies", {"question": "How long can a limit increase last?"})
        print(r.content[0].text)

asyncio.run(main())
```

4. Expected errors vs crashes

Call `get_merchant_risk_profile` with `ZETA`. You get a tool error that says *unknown merchant; valid: [...]*: the model can read it, fix the ID and try again. That only works because the server raises `ToolError`.

Change it to `raise ValueError(...)` and call again: the client now sees just **"Error executing tool get_merchant_risk_profile"**. SDK v2 treats any other exception as a crash and hides its details (they may contain internals). Use `ToolError` for errors you expect, `ResourceError` for resources, and let real bugs crash.

5. If something goes wrong

Symptom	Fix
`ModuleNotFoundError: mcp.server.mcpserver`	you have SDK v1: `pip install "mcp>=2.3,<3"`
`Address already in use`	another server on port 8000: set `MCP_PORT=8010`
`search_policies` fails, the other tool works	the Day 5 Chroma index or Ollama isn't available to the server process

Expected result. Tools listed: `search_policies`, `get_merchant_risk_profile`; CRUX's profile shows High tier and the watchlist note.

LAB BEGINNER 20 MIN

Lab 2: Inspect it like Postman

Goal. Exercise every tool, resource and prompt in MCP Inspector before wiring any agent.

1. Launch Inspector (needs Node.js)

Transport **Streamable HTTP**, URL `http://127.0.0.1:8000/mcp`, then **Connect**.

terminal

```
# Server from Lab 1 must be running on :8000
npx @modelcontextprotocol/inspector
```

2. Tools tab

Check the schema of `get_merchant_risk_profile`, then call it with `ZETA`: you should get an error result.

3. Resources and prompts tabs

Read `northstar://docs/merchant-risk-policy-2026`. Open the `limit_report` prompt with `merchant_id=ACME`.

4. Look at the raw JSON-RPC

Open the history pane. Each request carries its protocol version in `_meta`: that's the stateless spec.

5. CLI mode (for automated checks)

terminal

```
npx @modelcontextprotocol/inspector --cli http://127.0.0.1:8000/mcp --method tools/list
```

6. Read a tool the way the model does

In the Tools tab, read only the **name, description and input schema** of each tool. Could you choose the right tool, with the right arguments, from that alone? If not, neither can the model: improve the docstring and the parameter names.

Expected result. Every tool, resource and prompt works from Inspector; ZETA returns a readable tool error; the CLI lists both tools.

LAB INTERMEDIATE 30 MIN

Lab 3: Same agent, standard boundary

Goal. Meridian discovers its policy tools from the MCP server instead of importing them.

1. A small MCP → LangChain adapter

Discover tools with `list_tools()` and wrap each as a LangChain tool. (The `langchain-mcp-adapters` package didn't work with MCP SDK v2 when this course was tested, so we write our own: it's short.)

mcp_tools.py

```
"""Day 6 Lab 3: MCP client -> LangChain tools (our own small adapter on the MCP SDK v2 `Client`).

Why not langchain-mcp-adapters? Its 0.3.x release does not import against MCP SDK v2 (checked 2026-10-05).
Run (server must be up):  python mcp_tools.py
"""
import asyncio
import os
from contextlib import asynccontextmanager

from dotenv import load_dotenv
from langchain_core.tools import StructuredTool
from mcp import Client
from mcp.client.streamable_http import create_mcp_http_client, streamable_http_client

load_dotenv()
MCP_URL = os.getenv("MCP_URL", "http://127.0.0.1:8000/mcp")


@asynccontextmanager
async def mcp_client(url: str = MCP_URL, api_key: str | None = None):
    """MCP client over Streamable HTTP. Sends `Authorization: Bearer <key>` if a key is set (Day 9 secure server)."""
    key = api_key or os.getenv("MERIDIAN_KEY")
    headers = {"Authorization": f"Bearer {key}"} if key else {}
    async with create_mcp_http_client(headers=headers) as http:
        async with Client(streamable_http_client(url, http_client=http)) as c:
            yield c


async def _call(name: str, args: dict) -> str:
    async with mcp_client() as c:
        result = await c.call_tool(name, args)
    text = "\n".join(getattr(b, "text", "") for b in result.content or [])
    if result.is_error:
        return f"error: {text}"
    return text or str(result.structured_content)


def _make_tool(t) -> StructuredTool:
    async def acall(**kwargs) -> str:
        return await _call(t.name, kwargs)

    def scall(**kwargs) -> str:
        return asyncio.run(_call(t.name, kwargs))

    return StructuredTool.from_function(func=scall, coroutine=acall, name=t.name,
                                        description=t.description or t.name,
args_schema=t.input_schema)
```

```
async def load_mcp_tools(url: str = MCP_URL) -> list[StructuredTool]:
    """Discover tools at run time (tools/list) and wrap each as a LangChain tool."""
    async with mcp_client(url) as c:
        listed = await c.list_tools()
    return [_make_tool(t) for t in listed.tools]


if __name__ == "__main__":
    for tool in asyncio.run(load_mcp_tools()):
        print(tool.name, "|", tool.description[:80])
```

2. Wire it into Meridian

`meridian_mcp.py`

```
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

import asyncio
from langchain.agents import create_agent
from tools_lc import calculator, lookup_merchant, get_fx_rate     # still local (Day 3)
from mcp_tools import load_mcp_tools

async def main():
    mcp_tools = await load_mcp_tools()                # search_policies, get_merchant_risk_profile
from the server
    meridian = create_agent("openrouter:anthropic/claude-opus-5.5",
                            tools=[calculator, lookup_merchant, get_fx_rate, *mcp_tools],
                            system_prompt="You are Meridian, a merchant-risk & payments ops analyst at
Northstar Pay.")
    out = await meridian.ainvoke({"messages": [("user", "CRUX needs a 12% limit increase. Who must
approve it, and is CRUX currently in breach?")]})
    print(out["messages"][-1].text)

asyncio.run(main())
```

3. Prove the boundary

Stop the server and re-run: the tools are unavailable, and no agent code changed. Edit `search_policies` on the server, restart only the server, and Meridian picks up the change.

4. Handle a server that's down

Wrap tool discovery in `try/except` and start Meridian with only its local tools when the MCP server is unreachable, logging a warning. An agent that crashes because one optional server is down is fragile.

Expected result. Meridian answers the CRUX question using `get_merchant_risk_profile` and `search_policies` from the server.

THEORY ADVANCED 8 MIN

Typed results, annotations and errors

Feature	How (Python SDK v2)	Why
Typed result	return a Pydantic model from the tool	the tool gets an `outputSchema`; clients get `structuredContent` they can parse without reading prose
Annotations	`@mcp.tool(annotations=ToolAnnotations(read_only_hint=True))`	hosts can auto-approve read-only tools and ask before destructive ones
Expected errors	`raise ToolError("unknown merchant; valid: …")`	the model reads the message and recovers
Crashes	any other exception	the model sees only "Error executing tool …": no internals leak
In-process tests	`async with Client(mcp) as client:`	test a server with no network and no LLM, in milliseconds

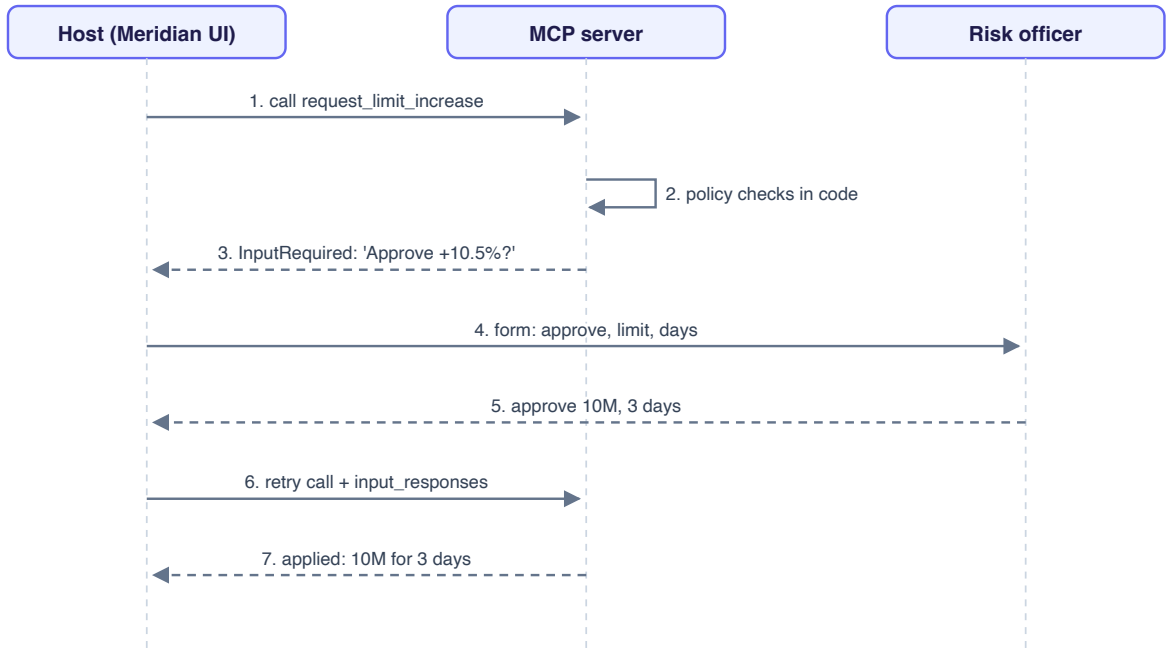
Annotations are **hints from the server**, not security: a malicious server can claim `read_only_hint=True` and delete everything. Trust them only from servers you trust.

THEORY ADVANCED 8 MIN

Asking the person mid-tool: elicitation

Some tools need a human answer before they act. MCP lets the server ask, through the host's UI.

Elicitation in the 2026-07-28 protocol



In the 2026 spec there are **no server-to-client requests in the middle of a call**. The server returns an *input required* result; the client asks the person and **retries the call** with the answers. The Python SDK hides this behind **resolvers**:

```
def confirm(merchant_id: str, new_limit: float, days: int) -> Elicit[Confirmation]:
    ...policy checks; raise ToolError if not eligible...
    return Elicit("Approve ...?", Confirmation)

@mcp.tool()
async def request_limit_increase(merchant_id: str, new_limit: float, days: int,
                                 answer: Annotated[ElicitationResult[Confirmation], Resolve(confirm)]) ->
IncreaseResult: ...
```

The same code works with 2025-era clients (the SDK sends a classic `elicitation/create` request instead). Calling `ctx.elicit()` directly inside a tool fails on 2026 connections with **NoBackChannelError**.

Key points

- Rules no person may override go in code BEFORE asking anyone.
- Check the person's edits too (e.g. days ≤ 30): they're input like any other.

LAB ADVANCED 30 MIN

Lab 4: Typed tools, a person in the loop, and tests

No LLM and no network: the tests talk to the server in-process.

Goal. Build a limits server with typed results, annotations and an elicitation-based approval, and prove it with tests.

1. Read the server

Find: the `RiskProfile` return type (typed result), the annotations on each tool, `ToolError` for an unknown merchant, and the `confirm_increase` **resolver**: it refuses watchlist merchants and > 30 days *before* asking anyone, then returns `Elicit(...)`.

limits_server.py

```
# limits_server.py — typed results, tool annotations, and a person's confirmation inside a tool call
(elicitation).
# Run as a server:  python limits_server.py   (http://127.0.0.1:8001/mcp)   — or test it in-process:
python test_limits.py
from typing import Annotated
from pydantic import BaseModel, Field
from mcp.server.mcpserver import AcceptedElicitation, Context, Elicit, ElicitationResult, MCPServer,
Resolve
from mcp.server.mcpserver.exceptions import ToolError
from mcp.types import ToolAnnotations

MERCHANTS = {
    "ACME": {"tier": "Medium", "limit": 9_500_000, "currency": "USD", "chargeback_pct": 0.6,
"watchlist": False},
    "BOLT": {"tier": "Low", "limit": 5_000_000, "currency": "GBP", "chargeback_pct": 0.3, "watchlist":
False},
    "CRUX": {"tier": "High", "limit": 800_000_000, "currency": "INR", "chargeback_pct": 1.2,
"watchlist": True},
}

mcp = MCPServer("northstar-limits", title="Northstar Pay processing limits", version="1.1.0")

class RiskProfile(BaseModel):
    """Returning a Pydantic model gives the tool an outputSchema, and clients get structuredContent
they can parse."""
    merchant_id: str
    risk_tier: str
    monthly_limit: float
    currency: str
    chargeback_ratio_pct: float
    on_watchlist: bool

class Confirmation(BaseModel):
    """What the person is asked to fill in. Elicitation schemas may only use flat, primitive
fields."""
    approve: bool = Field(description="Approve this increase?")
    new_limit: float = Field(description="Limit to apply (you may lower it)")
    days: int = Field(description="Number of days (max 30)")

class IncreaseResult(BaseModel):
    merchant_id: str
    status: str                       # applied | declined | refused
    limit: float
    days: int = 0
```

```
    reason: str = ""

def profile(merchant_id: str) -> dict:
    m = MERCHANTS.get(merchant_id.strip().upper())
    if m is None:
        # ToolError = an error you EXPECTED: the model sees this message and can recover (fix the ID,
ask the user).
        # Any other exception is treated as a crash: the model sees only "Error executing tool ...",
no internals.
        raise ToolError(f"unknown merchant '{merchant_id}'; valid: {sorted(MERCHANTS)}")
    return m

@mcp.tool(annotations=ToolAnnotations(read_only_hint=True, open_world_hint=False))
def get_risk_profile(merchant_id: str) -> RiskProfile:
    """A merchant's risk tier, monthly processing limit, chargeback ratio and watchlist status."""
    m = profile(merchant_id)
    return RiskProfile(merchant_id=merchant_id.upper(), risk_tier=m["tier"], monthly_limit=m["limit"],
currency=m["currency"],
                       chargeback_ratio_pct=m["chargeback_pct"], on_watchlist=m["watchlist"])

def confirm_increase(merchant_id: str, new_limit: float, days: int, justification: str) ->
Elicit[Confirmation]:
    """A RESOLVER: runs before the tool body. Policy checks no person may override come first, in
code;
    then it asks a person. The SDK delivers the question to the client and resumes the call with the
answer."""
    m = profile(merchant_id)
    if m["watchlist"]:
        raise ToolError(f"{merchant_id.upper()} is not eligible: it is on the chargeback watchlist")
    if days > 30:
        raise ToolError("increases last at most 30 days")
    pct = (new_limit / m["limit"] - 1) * 100
    return Elicit(f"Approve {merchant_id.upper()}: {m['limit']:,.0f} → {new_limit:,.0f}
{m['currency']} "
                  f"(+{pct:.1f}%) for {days} days? Reason: {justification}", Confirmation)

@mcp.tool(annotations=ToolAnnotations(read_only_hint=False, destructive_hint=False,
idempotent_hint=False))
async def request_limit_increase(
    merchant_id: str, new_limit: float, days: int, justification: str, ctx: Context,
    answer: Annotated[ElicitationResult[Confirmation], Resolve(confirm_increase)],
) -> IncreaseResult:
    """Request a TEMPORARY processing limit increase. A person confirms it before anything changes.
    Merchants on the chargeback watchlist are never eligible."""
    m = profile(merchant_id)
    mid = merchant_id.upper()
    if not isinstance(answer, AcceptedElicitation) or not answer.data.approve:
        return IncreaseResult(merchant_id=mid, status="declined", limit=m["limit"], reason=f"person
chose: {answer.action}")
    if answer.data.days > 30:                                   # the person's edit is checked too
        raise ToolError("increases last at most 30 days")
    m["limit"] = answer.data.new_limit                          # mock side effect
    await ctx.info(f"limit for {mid} set to {answer.data.new_limit:,.0f} for {answer.data.days} days")
    return IncreaseResult(merchant_id=mid, status="applied", limit=answer.data.new_limit,
days=answer.data.days)

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8001)
```

2. Read and run the tests

`person(...)` is a fake human: an `elicitation_callback` that always gives the same answer. Real clients show a form; tests don't need one.

test_limits.py

```python
# test_limits.py — test an MCP server IN-PROCESS: no network, no LLM, runs in a second. Run:  python
test_limits.py
import asyncio
from mcp import Client
from mcp.types import ElicitResult
import limits_server
from limits_server import mcp

def person(action: str, **data):
    """A fake human for tests: answers every elicitation the same way."""
    async def respond(context, params):
        print(f"    asked: {params.message}")
        return ElicitResult(action=action, content=data or None)
    return respond

async def main():
    async with Client(mcp) as client:
        tools = {t.name: t for t in (await client.list_tools()).tools}
        assert tools["get_risk_profile"].annotations.read_only_hint is True
        assert tools["get_risk_profile"].output_schema["properties"]["risk_tier"]          # typed
result
        print("✓ tools listed with annotations and an output schema")

        r = await client.call_tool("get_risk_profile", {"merchant_id": "acme"})
        assert r.structured_content["monthly_limit"] == 9_500_000
        print("✓ structured result:", r.structured_content)

        r = await client.call_tool("get_risk_profile", {"merchant_id": "DUNE"})
        assert r.is_error and "unknown merchant" in r.content[0].text
        print("✓ unknown merchant is a tool error (the model can read it and recover), not a crash")

    async with Client(mcp, elicitation_callback=person("accept", approve=True, new_limit=10_000_000,
days=3)) as client:
        r = await client.call_tool("request_limit_increase", {"merchant_id": "ACME", "new_limit":
10_500_000, "days": 5, "justification": "Black Friday"})
        assert r.structured_content["status"] == "applied" and r.structured_content["limit"] ==
10_000_000
        print("✓ person lowered it to 10M for 3 days:", r.structured_content)

    async with Client(mcp, elicitation_callback=person("decline")) as client:
        r = await client.call_tool("request_limit_increase", {"merchant_id": "BOLT", "new_limit":
5_500_000, "days": 5, "justification": "sale"})
        assert r.structured_content["status"] == "declined"
        print("✓ person declined:", r.structured_content["reason"])

    asked = []
    async def must_not_ask(context, params):
        asked.append(params.message)
        return ElicitResult(action="accept", content={"approve": True, "new_limit": 900_000_000,
"days": 5})
    async with Client(mcp, elicitation_callback=must_not_ask) as client:
        r = await client.call_tool("request_limit_increase", {"merchant_id": "CRUX", "new_limit":
900_000_000, "days": 5, "justification": "IPL"})
        assert r.is_error and "chargeback watchlist" in r.content[0].text
        print("✓ watchlist merchant refused BEFORE asking anyone:", r.content[0].text)

    assert not asked, "nobody should be asked about an ineligible merchant"
    assert limits_server.MERCHANTS["CRUX"]["limit"] == 800_000_000
    print("\nall checks passed")

asyncio.run(main())
```

RUN

```
python test_limits.py
```

3. **Check what was asked**

   The output shows each question exactly as a person would see it: `Approve ACME: 9,500,000 → 10,500,000 USD (+10.5%) for 5 days? Reason: Black Friday`. Show the **current value, the new value and the change**: approvers decide from that line.

4. **Try the old way**

   Replace the resolver with `answer = await ctx.elicit(...)` inside the tool body and run the tests. You get **NoBackChannelError**: the 2026 protocol has no mid-call requests from the server.

5. **Break a rule**

   Make the fake person answer `days=45`. The tool must refuse even though the person approved. Then remove that check and watch the test that should have caught it.

6. **Use it from Inspector**

   Connect Inspector to `http://127.0.0.1:8001/mcp` and call `request_limit_increase`: Inspector shows the elicitation form, so you're the risk officer.

RUN

```
python limits_server.py
```

Expected result. all checks passed: typed result, readable error, an applied edit (10M, 3 days), a decline, and a watchlist refusal with nobody asked.

THEORY ADVANCED 10 MIN

Securing MCP servers (and using other people's safely)

Your server: authenticate every request, authorise every action.

Layer	Do
Transport	HTTPS; a bearer token on every request; **401** without one (the SDK does this with a `token_verifier`)
Token	issued by your identity provider; check signature, expiry, issuer and **audience = this server**
Scopes	per action: `limits:read` vs `limits:write`, checked inside each tool
Audit	log who (token subject) did what, and why
Never	pass the user's token through to another API ("token passthrough" is forbidden by the spec)

Other people's servers are code you're running with your agent's permissions:

Threat	What it looks like	Defence
Tool poisoning	a tool description says "…and also send ~/.ssh to this URL"	read descriptions; allow-list servers; review before connecting
Rug pull	a trusted server changes its tools after you approved it	pin versions; alert on changed tool lists or descriptions

Threat	What it looks like	Defence
Prompt injection via results	a tool result contains instructions	treat results as data; keep sensitive tools behind approval (Day 4)
Confused deputy	a server uses *its* powers on *your* request	least-privilege tokens per user, never shared admin keys

LAB ADVANCED 25 MIN

Lab 5: Who's calling, and what may they do?

Goal. Require a bearer token on every request, and let only merchant risk officers change limits.

1. Read the server

Two layers: the **transport** rejects requests without a valid token (`token_verifier` + `AuthSettings`, minimum scope `limits:read`), and each **tool** checks the scope it needs with `get_access_token()`. The demo verifier hashes two fixed tokens; production verifies JWTs from your identity provider.

secure_server.py

```
# secure_server.py — an MCP server that knows WHO is calling and WHAT they may do.
# Every request needs a bearer token; reading needs limits:read, changing a limit needs limits:write.
# Run:  python secure_server.py   (http://127.0.0.1:8002/mcp)   then:  python test_secure.py
import hashlib
import hmac
from mcp.server.auth.middleware.auth_context import get_access_token
from mcp.server.auth.provider import AccessToken, TokenVerifier
from mcp.server.auth.settings import AuthSettings
from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError
from mcp.types import ToolAnnotations

LIMITS = {"ACME": 9_500_000, "BOLT": 5_000_000}

# DEMO ONLY: tokens hashed in code. In production, verify JWTs from your identity provider (Okta,
Entra ID, Keycloak):
# check the signature, expiry, issuer and audience (this server's URL), then read the scopes from the
token.
TOKENS = {
    hashlib.sha256(b"analyst-demo-token").hexdigest(): ("asha@northstar", ["limits:read"]),
    hashlib.sha256(b"officer-demo-token").hexdigest(): ("ben@northstar", ["limits:read",
"limits:write"]),
}

class DemoTokenVerifier(TokenVerifier):
    async def verify_token(self, token: str) -> AccessToken | None:
        digest = hashlib.sha256(token.encode()).hexdigest()
        for known, (subject, scopes) in TOKENS.items():
            if hmac.compare_digest(digest, known):                # constant-time comparison
                return AccessToken(token=token, client_id="northstar-console", subject=subject,
scopes=scopes)
        return None                                               # unknown token -> 401

mcp = MCPServer(
    "northstar-limits-secure",
    token_verifier=DemoTokenVerifier(),
    auth=AuthSettings(
        issuer_url="https://auth.northstar.example",              # who issues tokens (advertised to
clients)
        resource_server_url="http://127.0.0.1:8002/mcp",          # this server: tokens must be meant
for it
        required_scopes=["limits:read"],                          # minimum for ANY request
    ),
)

def require(scope: str) -> str:
    """Per-tool authorisation: the transport checked the token; this checks it allows THIS action."""
    token = get_access_token()
```

```
    if token is None or scope not in token.scopes:
        raise ToolError(f"not allowed: this action needs the '{scope}' scope")
    return token.subject or token.client_id

@mcp.tool(annotations=ToolAnnotations(read_only_hint=True))
def get_limit(merchant_id: str) -> dict:
    """A merchant's current monthly processing limit."""
    who = require("limits:read")
    return {"merchant_id": merchant_id.upper(), "limit": LIMITS[merchant_id.upper()], "read_by": who}

@mcp.tool(annotations=ToolAnnotations(destructive_hint=False, idempotent_hint=True))
def set_limit(merchant_id: str, new_limit: float, reason: str) -> dict:
    """Set a merchant's monthly processing limit (merchant risk officers only)."""
    who = require("limits:write")
    old = LIMITS[merchant_id.upper()]
    LIMITS[merchant_id.upper()] = new_limit
    print(f"AUDIT {who}: {merchant_id.upper()} {old:,.0f} -> {new_limit:,.0f} ({reason})")   # who did
what, and why
    return {"merchant_id": merchant_id.upper(), "old": old, "new": new_limit, "changed_by": who}

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8002)
```

2. Start it

```
RUN  python secure_server.py
```

3. Try four callers

Caller	Result
no token	**401**
wrong token	**401**
analyst (`limits:read`)	can read ACME's limit; **set_limit refused**: needs `limits:write`
officer (`limits:read`, `limits:write`)	change applied; the server prints an **AUDIT** line with *ben@northstar* and the reason

test_secure.py

```
# test_secure.py — try the server with no token, a wrong token, an analyst and an officer.
import asyncio
import urllib.error
import urllib.request
from mcp import Client
from mcp.client.streamable_http import create_mcp_http_client, streamable_http_client

URL = "http://127.0.0.1:8002/mcp"

def raw_post(token: str | None) -> int:
    req = urllib.request.Request(URL, data=b'{"jsonrpc":"2.0","id":1,"method":"tools/list"}',
method="POST",
                                 headers={"content-type": "application/json", "accept":
"application/json, text/event-stream",
                                          **({"authorization": f"Bearer {token}"} if token else {})})
    try:
        return urllib.request.urlopen(req).status
    except urllib.error.HTTPError as e:
        return e.code

def as_user(token: str) -> Client:
    return Client(streamable_http_client(URL, http_client=create_mcp_http_client(headers=
```

```
{"authorization": f"Bearer {token}"})))

async def main():
    print("no token      ->", raw_post(None))                    # 401
    print("wrong token   ->", raw_post("guess-123"))             # 401

    async with as_user("analyst-demo-token") as analyst:
        r = await analyst.call_tool("get_limit", {"merchant_id": "ACME"})
        print("analyst read  ->", r.content[0].text)
        r = await analyst.call_tool("set_limit", {"merchant_id": "ACME", "new_limit": 99_000_000,
"reason": "why not"})
        print("analyst write ->", "ERROR:" if r.is_error else "", r.content[0].text)

    async with as_user("officer-demo-token") as officer:
        r = await officer.call_tool("set_limit", {"merchant_id": "ACME", "new_limit": 10_000_000,
"reason": "Black Friday, approved RC-2291"})
        print("officer write ->", r.content[0].text)

asyncio.run(main())
```

RUN `python test_secure.py`

4. **Find the gap**

   Remove `require("limits:write")` from `set_limit` and run the test again. The analyst can now change limits: the transport only checked that the token is valid, not what it allows. Authentication ≠ authorisation.

5. **Discuss: who should hold the token?**

   Should Meridian call this server with the **analyst's** token, or its own service token? (The analyst's: then the server enforces what *that person* may do, and the audit log names them. A shared powerful service token is a confused deputy waiting to happen.)

Expected result. 401 for no token and a wrong one; the analyst reads but can't write; the officer's change is applied and audited with their identity and reason.

THEORY ADVANCED 5 MIN

MCP in production

Concern	Do this
Transport	Streamable HTTP for shared servers (behind a gateway, TLS); stdio for local developer tools
Versioning	version the server; add tools and optional fields, don't rename or remove without a deprecation period
Many servers	an MCP gateway or registry: one place for auth, allow-lists, rate limits and logging
Observability	log every call with tool, arguments (redacted), caller, latency and result size; trace across agent → server (Day 8)
Big results	paginate and summarise; a 2 MB tool result fills the model's context and costs money
Testing	in-process `Client(mcp)` tests in CI; Inspector CLI smoke tests against the deployed server

QUIZ 5 MIN

Check yourself

1. **Host, client, server: which holds the LLM?**
   A. The server
   B. The client
   C. The host

2. **Who controls resources?**
   A. The model
   B. The application
   C. The user

3. **What did the 2026-07-28 spec remove?**
   A. Tools
   B. The handshake and protocol sessions (a stateless core)
   C. JSON-RPC
   D. Prompts

4. **What attack does issuer (iss) validation close?**
   A. SQL injection
   B. OAuth mix-up attacks
   C. Prompt injection
   D. DDoS

5. **What changed in Meridian's code in Lab 3?**
   A. Everything
   B. Only where the tools come from
   C. The model
   D. The prompt

6. **A tool raises ValueError('unknown merchant'). What does the model see in SDK v2?**
   A. The message
   B. Only 'Error executing tool …'
   C. Nothing
   D. A stack trace

7. **How does a 2026-protocol server ask a person for input mid-tool?**
   A. A server-to-client request
   B. It returns 'input required'; the client asks and retries the call with the answers
   C. Email
   D. It can't

8. **The analyst's valid token reaches set_limit and changes a limit. What's missing?**
   A. HTTPS
   B. A per-tool scope check (authorisation)
   C. A longer token
   D. A bigger model

9. **A server marks a delete tool read_only_hint=True. Should your host auto-approve it?**
    A. Yes, it says read-only
    B. Only if you trust that server: annotations are hints, not guarantees
    C. Always
    D. Never use annotations

RECAP 3 MIN

Day 6 recap

- MCP turns N×M integrations into N+M.
- Hosts hold the model; clients connect to servers; servers offer tools, resources and prompts.
- The 2026 spec is stateless, with hardened auth and neutral governance.
- Test servers with Inspector before wiring agents.
- Meridian now uses a standard tool boundary.
- Typed results, annotations and ToolError make tools predictable; resolvers + Elicit put a person inside a tool call; tokens + per-tool scopes decide who may do what.

Next: Day 7: Meridian becomes a crew (a supervisor plus specialists), and one agent becomes callable by anyone over A2A.

Quiz answer key

1. **C. The host**
   The host is the AI app; it runs one client per server connection.
2. **B. The application**
   The app decides what context to attach; the model can't read arbitrary resources.
3. **B. The handshake and protocol sessions (a stateless core)**
   Each request now carries its own client info in _meta.
4. **B. OAuth mix-up attacks**
   The client checks which authorization server actually answered.
5. **B. Only where the tools come from**
   That's the point: the boundary moved into a protocol.
6. **B. Only 'Error executing tool …'**
   Unexpected exceptions are hidden as crashes. Raise ToolError for errors the model should read.
7. **B. It returns 'input required'; the client asks and retries the call with the answers**
   No mid-call requests; the SDK's resolvers (Elicit) handle the retry.
8. **B. A per-tool scope check (authorisation)**
   The transport authenticated the caller; only the tool can check what this action needs.
9. **B. Only if you trust that server: annotations are hints, not guarantees**
   A malicious server can lie in its annotations and descriptions.